



Actividad Práctica – Semana 6.

ERD de tu caso

Cristhian Gaitán Guzmán

Facultad de Ingeniería, Corporación Universitaria del Huila (CORHUILA)

Ciencia de Datos (Cód. 69109) [Pénsum 40D, Grupo 1]

Unidad 2 · Modelamiento, transformación y conexión de datos · Semana 6,

Corte 2

Jesús Ariel Gonzáles Bonilla

Octubre de 2026





1. Caso

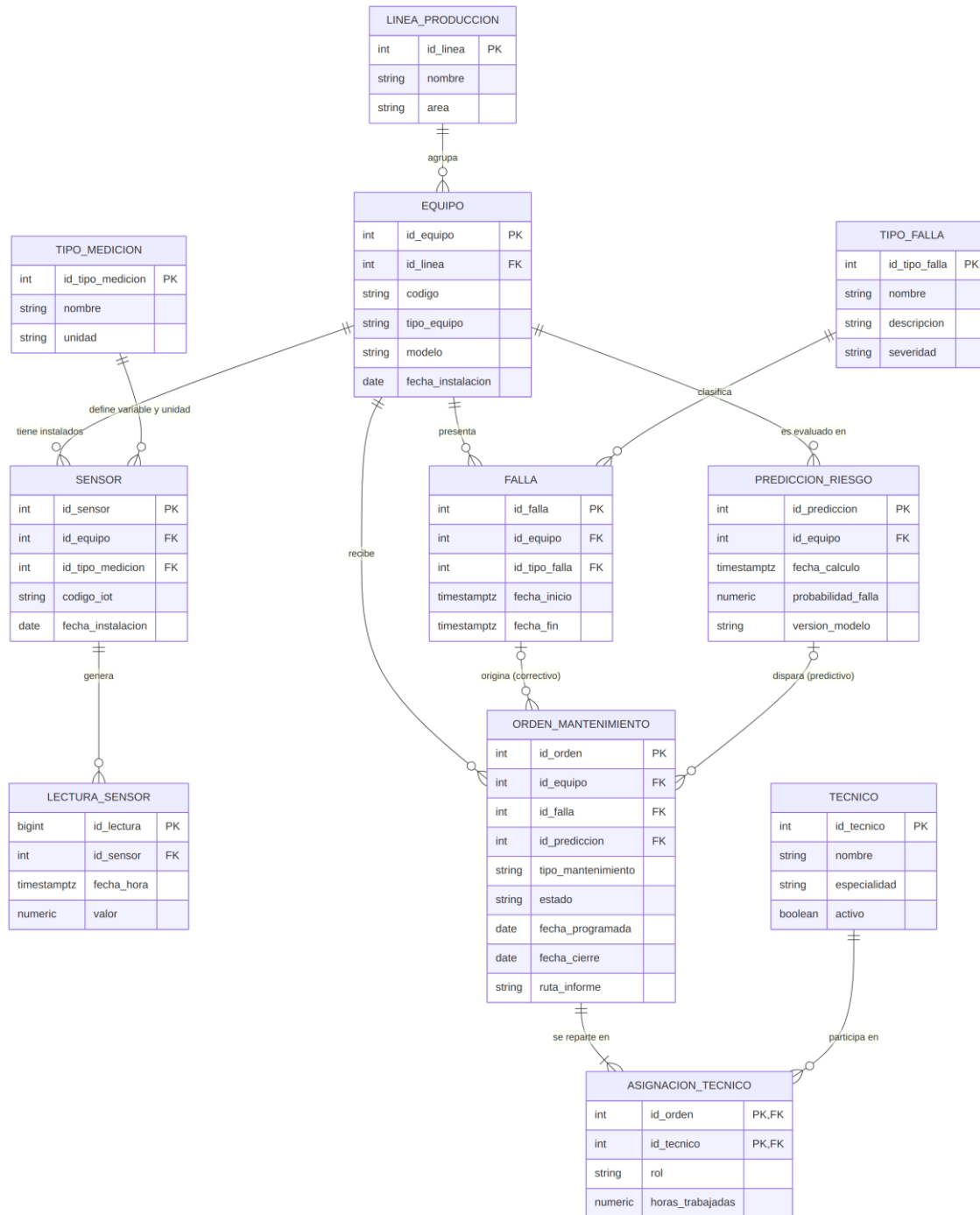
Seguimos con el mismo caso de las semanas anteriores: el **mantenimiento predictivo de motores y equipos industriales en una planta manufacturera de Siemens**. En la Semana 1 planteamos la pregunta (¿cómo usar los datos de funcionamiento para anticipar fallas?), en la Semana 4 hicimos el inventario de datos (temperatura, vibración, consumo eléctrico, presión, horas de funcionamiento, historial de fallas, registros de mantenimiento, eventos del sistema e informes técnicos) y definimos un resultado esperado: un nivel de riesgo por equipo con una acción asociada.

Esta semana damos el paso que sigue: decidir **cómo se guardan y se conectan esos datos** para que la pregunta se pueda responder con consultas. El ERD que se presenta es ilustrativo (no corresponde a la base de datos real de Siemens), pero cada entidad sale directamente de un dato ya identificado en el inventario de la Semana 4.

2. ERD del caso (entidades, PK/FK y relaciones)

2.1 Diagrama

El diagrama sigue el modelo entidad-relación de Chen [3] y está escrito en Mermaid [5], que GitHub renderiza directamente al abrir este archivo. La notación de patas de gallo se lee así: $|| =$ uno y solo uno, $|o =$ cero o uno, $o\{ =$ cero o muchos, $| \{ =$ uno o muchos.





2.2 Entidades, llaves primarias y foráneas

El modelo tiene **11 entidades**: cuatro catálogos que evitan repetir texto (línea, tipo de medición, tipo de falla y técnico), y siete tablas que registran lo que sucede en la planta.

Entidad	PK	FK	Qué representa en el caso
linea_produccion	id_linea	—	Línea o área de la planta donde trabaja un grupo de equipos.
equipo	id_equipo	id_linea	Cada motor o equipo monitoreado (p. ej. Motor A).
tipo_medicion	id_tipo_medicion	—	Variable medida y su unidad: temperatura (°C), vibración (mm/s), corriente (A), presión (bar), horas de operación (h).
sensor	id_sensor	id_equipo, id_tipo_medicion	Dispositivo IoT instalado en un equipo que mide una variable.
lectura_sensor	id_lectura	id_sensor	Una medición puntual: qué sensor, en qué instante y con qué valor.
tipo_falla	id_tipo_falla	—	Catálogo de fallas (p. ej. rodamiento desgastado) con su severidad.
falla	id_falla	id_equipo, id_tipo_falla	Evento de falla ocurrido en un equipo, con fecha de inicio y de fin.
prediccion_riesgo	id_prediccion	id_equipo	Resultado del modelo predictivo: probabilidad de falla de un equipo en una fecha.
orden_mantenimiento	id_orden	id_equipo, id_falla*, id_prediccion*	Intervención programada o ejecutada (preventiva, predictiva o correctiva).
tecnico	id_tecnico	—	Persona que ejecuta mantenimientos.
asignacion_tecnico	(id_orden, id_tecnico)	id_orden, id_tecnico	Tabla intermedia de la relación N:M entre órdenes y técnicos.

* Opcionales: `id_falla` solo se llena en mantenimientos correctivos e `id_prediccion` solo en los predictivos.

2.3 Relaciones

Relación	Cardinalidad	Cómo se lee
linea_produccion → equipo	1:N	Una línea agrupa varios equipos; cada equipo pertenece a una sola línea.
equipo → sensor	1:N	Un equipo puede tener varios sensores (temperatura, vibración, corriente...).
tipo_medicion → sensor	1:N	Muchos sensores miden la misma variable con la misma unidad.
sensor → lectura_sensor	1:N	Un sensor produce muchas lecturas a lo largo del tiempo.
equipo → falla y tipo_falla → falla	1:N	Un equipo acumula un historial de fallas; cada falla es de un tipo del catálogo.
equipo → prediccion_riesgo	1:N	Al equipo se le recalcula el riesgo periódicamente.
equipo → orden_mantenimiento	1:N	Un equipo recibe muchas órdenes a lo largo de su vida.
falla → orden_mantenimiento	0..1:N	Una falla puede originar órdenes correctivas; una orden no siempre viene de una falla.
prediccion_riesgo → orden_mantenimiento	0..1:N	Una predicción de riesgo alto puede disparar una orden predictiva.
orden_mantenimiento ↔ tecnico	N:M	Una orden puede requerir varios técnicos y un técnico atiende muchas órdenes. Se resuelve con asignacion_tecnico, cuya PK compuesta (id_orden, id_tecnico) está formada por las dos FK, y que además guarda rol y horas_trabajadas, datos que no pertenecen ni a la orden ni al técnico por separado.

La razón de elegir esta N:M es que ocurre de verdad en una planta: una intervención sobre un motor grande suele requerir un técnico mecánico y uno eléctrico, y cada uno de ellos trabaja en muchas órdenes durante el mes. Sin tabla intermedia, habría que meter una lista de técnicos dentro de una celda de orden_mantenimiento, lo que rompe la primera forma normal.

El diseño completo en SQL (con tipos, restricciones CHECK, índices y una vista) está en schema.sql; se explica en el README.md de esta carpeta.



3. Relacional o NoSQL

Decisión: usaría un enfoque relacional como núcleo del sistema (por ejemplo PostgreSQL [4]) y dejaría a un lado los datos que realmente no encajan en tablas.

Los motivos principales son estos:

- **Los datos centrales son estructurados y están muy conectados.** Equipos, sensores, fallas, órdenes y técnicos se relacionan entre sí todo el tiempo, y la pregunta del caso se responde precisamente cruzándolos: ¿qué lecturas tenía el motor antes de cada falla?, ¿qué órdenes se generaron a partir de una predicción de riesgo alto? Eso son JOIN, justo lo que el modelo relacional hace mejor [1].
- **La integridad importa.** Una orden de mantenimiento no puede apuntar a un equipo que no existe, ni a una falla de otro equipo. Las llaves foráneas y las restricciones lo garantizan desde la base de datos, sin depender de que cada aplicación lo valide bien.
- **Hay una relación N:M y transacciones.** Asignar técnicos a una orden y cerrarla son operaciones que deben ser consistentes (todo se guarda o nada), algo que las bases relacionales manejan de forma nativa.
- **Es lo que viene después.** La Semana 7 trabaja consultas SQL (WHERE, JOIN, GROUP BY) y su equivalente en pandas, y este esquema sirve directamente como base.



Un enfoque NoSQL no se descarta del todo. En la Semana 4 clasificamos algunos datos como semiestructurados y no estructurados, y esos sí se acomodan mejor fuera de las tablas:

Dato del inventario (Semana 4)	Dónde viviría	Razón
Lecturas de sensores IoT a alta frecuencia	Relacional al inicio; si el volumen crece mucho, una base de series de tiempo o la propia plataforma de monitoreo industrial [6]	Cantidad enorme de registros que casi nunca se modifican y se consultan por rango de fechas.
Eventos y alertas del sistema (JSON)	Almacén de documentos	Estructura variable según el dispositivo; forzarla a columnas fijas sería frágil.
Informes técnicos de mantenimiento	Almacenamiento de archivos o documentos; en la tabla solo queda ruta_informe	Son texto libre, no tienen un esquema que normalizar.

Es decir, la propuesta es **relacional como columna vertebral** y **NoSQL como complemento** para lo semiestructurado y no estructurado. Una salvedad honesta: en la Semana 4 se concluyó que el caso es un *potencial* Big Data, que depende de la escala real de la planta. Si la tabla `lectura_sensor` llegara a crecer a miles de millones de filas, habría que particionarla por fecha o moverla a un motor especializado; para un diagnóstico a escala de una planta, un único esquema relacional es suficiente y más simple de mantener.

4. Normalización: qué se evitó repetir

La normalización consiste en organizar los datos para que cada hecho se guarde una sola vez, de forma que corregirlo no obligue a cambiarlo en muchos lugares [2]. Para verlo con claridad, imaginemos el punto de partida más natural: una sola hoja de cálculo de mantenimiento, como esta:

equipo	línea	temp 1	temp 2	unidad temp	falla	severidad falla	técnico 1	técnico 2	horas parada
Motor A	Línea 1	78.5	81.0	°C	Rodamiento desgastado	alta	Ana	Luis	4.5

Esta hoja repite la línea en cada equipo, la unidad en cada fila, la severidad en cada falla, y limita los técnicos a dos columnas. Se descompuso así:

Forma normal	Problema en la hoja original	Cómo se resolvió en el ERD
1FN (valores atómicos, sin grupos repetidos)	Columnas temp_1, temp_2, técnico_1, técnico_2: ¿qué pasa con la tercera lectura o el tercer técnico?	Cada medición es una fila en lectura_sensor; cada técnico asignado es una fila en asignacion_tecnico.
2FN (cada atributo depende de toda la llave)	En una tabla orden-técnico, el nombre o la especialidad del técnico dependería solo de id_tecnico, no de la pareja completa.	Los datos del técnico viven en tecnico; asignacion_tecnico solo guarda lo que depende de la pareja orden-técnico (rol, horas_trabajadas).
3FN (sin dependencias entre atributos que no son llave)	unidad_temp depende de la variable, no del equipo; severidad_falla depende del tipo de falla; la línea depende del equipo.	tipo_medicion guarda variable y unidad una vez; tipo_falla guarda descripción y severidad una vez; linea_produccion guarda los datos de cada línea una vez.

En concreto, esto es lo que se evitó repetir:

Dato	Dónde se guarda (una sola vez)	Qué se evita
Unidad de medida (°C, mm/s, A, bar, h)	tipo_medicion	Escribir «°C» en millones de lecturas y tener «°C» y «C» mezclados.
Severidad y descripción de cada tipo de falla	tipo_falla	Describir de nuevo «rodamiento desgastado» en cada falla.



Dato	Dónde se guarda (una sola vez)	Qué se evita
Nombre y especialidad del técnico	tecnico	Que un cambio de nombre exija actualizar cientos de órdenes.
Datos de la línea de producción	linea_produccion	Repetir la línea en cada equipo.
Horas de parada de una falla	No se almacena: se calcula con <code>fecha_fin - fecha_inicio</code>	Que la duración quede inconsistente con las fechas.
Nivel de riesgo (alto/medio/bajo)	No se almacena: la vista <code>v_riesgo_actual</code> lo deriva de <code>probabilidad_falla</code>	Que el nivel diga «alto» mientras la probabilidad dice 0.2.
Horas acumuladas de operación	Son una medición más (<code>tipo_medicion = horas_operacion</code>)	Mantener una columna que hay que actualizar a mano en equipo.

Hay además una decisión de diseño que refuerza la consistencia: `orden_mantenimiento` incluye `id_equipo` y también puede apuntar a una falla o a una predicción. Para que no se pueda asociar una orden del Motor A con una falla del Motor B, las llaves foráneas hacia `falla` y `prediccion_riesgo` son **compuestas** (`id_falla`, `id_equipo`), de modo que la propia base de datos rechaza esa combinación. Esto se comprobó con una inserción de prueba (ver `README.md`).

Un punto que no se normalizó más: `especialidad` en `tecnico` y `tipo_equipo` en `equipo` son texto libre y podrían ser catálogos propios. Se dejaron así para no inflar el modelo en una actividad formativa; sería la primera mejora si el sistema creciera.

5. Conclusión

El ERD convierte el inventario de datos de la Semana 4 en un modelo concreto: cada dato listado allí tiene ahora una tabla donde vivir y una llave que lo conecta con los demás. La tabla `prediccion_riesgo` y la vista `v_riesgo_actual` materializan el resultado esperado que



planteamos entonces (Motor A en riesgo alto, Motor B medio, Motor C bajo) y la tabla `orden_mantenimiento` cierra el ciclo, porque permite registrar la decisión que se toma a partir de ese riesgo, tal como lo planteaba la pregunta original de la Semana 1.

Elegir relacional como núcleo y NoSQL como complemento es coherente con lo que ya sabíamos del caso: los datos estructurados y relacionados son el corazón del análisis, mientras que los eventos del sistema y los informes técnicos son la parte semiestructurada y no estructurada. La normalización, por último, no es un fin en sí misma: evita que un mismo dato aparezca escrito de varias formas, y eso importa mucho cuando el modelo predictivo depende de que las lecturas sean confiables (la «veracidad» de las 5 V). Este esquema será la base para las consultas SQL y el análisis en pandas de la Semana 7.



6. Referencias

- [1] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.
- [2] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed., Courant Computer Science Symposia Series, vol. 6. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.
- [3] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976.
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [5] Mermaid, "Entity Relationship Diagrams," *Mermaid Documentation*, 2026. [Online]. Available: <https://mermaid.js.org/syntax/entityRelationshipDiagram.html>
- [6] Siemens AG, "Insights Hub: plataforma de IoT industrial en la nube (anteriormente MindSphere)," Siemens Digital Industries Software, 2026. [Online]. Available: <https://www.siemens.com/en-us/products/insights-hub/>

7. Entrega

- **Carpeta:** 06-week/ en el fork del repositorio de la clase.
- **Archivos:** c2-Activity_Semana6_CGG.md (este documento), README.md (README técnico), schema.sql (DDL), erd.mmd (fuente del diagrama).
- **Comando:** `git add .` `git commit -m "Entrega semana 06"` `git push`.
- **Repositorio de perfil:** con el bloque CONFIG (FULL_NAME + GITHUB_USER) actualizado, como pide la guía.